

Project Nurse Simulator

DAW 2
2026/2027

pr4

Ignacio Breñas Muñoz
Christopher Roger Tito Farez
Zehao Yin
Gorka Ramirez

stucom

Índice

https://github.com/Ignaciobrenas/Nurse-Simulator

Contenido

1. Explicación del proyecto 4
1.1. Descripción general 4
1.2. Qué cambia respecto a la pr03 4
2. Cómo organizamos el trabajo 5
2.1. Metodología con GitHub 5
2.2. Evidencias de la organización 6
2.2.1. Tablero de GitHub Project 6
2.2.2. Issues y ramas 7
2.2.3. Pull requests y commits 8
3. Parte individual 10
3.1. Configuración y parametrización de Doctrine 10
3.1.1. Instalación 10
3.1.2. Parametrización de la conexión 10
3.2. Entidad Nurse 10
3.3. Repositorio NurseRepository 10
3.4. Enlaces a los repositorios individuales 11
4. Parte grupal 12
4.1. Configuración de Doctrine en el repositorio grupal 12
4.2. Base de datos en Supabase (PostgreSQL) 12
4.2.1. Por qué Supabase 12
4.2.2. PostgreSQL también en local 12
4.3. Adaptación de los endpoints 13
4.3.1. Read all nurses: GET /nurse/index 13
4.3.2. Nurse login: POST /nurse/login 13
4.3.3. Find nurse by name: GET /nurse/name/{name} 13
5. Diseño de la base de datos y pruebas 14
5.1. Diseño final de la base de datos 14
5.2. Comprobación con Postman 16
5.2.1. GET /nurse/index 17
5.2.2. POST /nurse/login 18

5.2.3. GET /nurse/name/{name} 20

1. Explicación del proyecto

1.1. Descripción general

https://github.com/Ignaciobrenas/Nurse-Simulator

Nurse Simulator es una API REST que simula la gestión de enfermeros de un hospital. Está desarrollada con Symfony 7.4 y PHP 8.2+, y expone endpoints HTTP que responden en formato JSON.

Este documento recoge la entrega de la **pr04: Integración backend con base de datos** (DAM MP0616), en la que el proyecto deja de leer los datos de un fichero y pasa a usar una base de datos a través de Doctrine, la herramienta de mapeo objeto-relacional de Symfony.

1.2. Qué cambia respecto a la pr03

En la pr03 los datos de los enfermeros se leían del fichero `data/nurses.json`. En la pr04 ese fichero deja de utilizarse: los tres endpoints consultan ahora una base de datos PostgreSQL alojada en Supabase mediante la entidad `Nurse` y su `NurseRepository`.

Endpoint	Antes (pr03)	Ahora (pr04)
`GET /nurse/index`	Lectura de `nurses.json`	Consulta a la base de datos
`POST /nurse/login`	Búsqueda en `nurses.json`	Consulta a la base de datos
`GET /nurse/name/{name}`	Búsqueda en `nurses.json`	Consulta a la base de datos

2. Cómo organizamos el trabajo

2.1. Metodología con GitHub

El equipo mantiene la misma organización que en la pr03:

- Cada tarea es un **issue**, gestionado en un tablero de **GitHub Project** con las columnas Todo, In progress y Done.
- Cada issue tiene su **rama propia** (`<n°>-<descripción>`).
- Los cambios entran por **pull request**, con revisión previa, hacia `staging` y finalmente `main`.
- Los **commits** siguen la convención `usuario/Tipo: descripción #issue`.

2.2. Evidencias de la organización

2.2.1. Tablero de GitHub Project

Estado de las tareas de la pr04 en el tablero del proyecto.

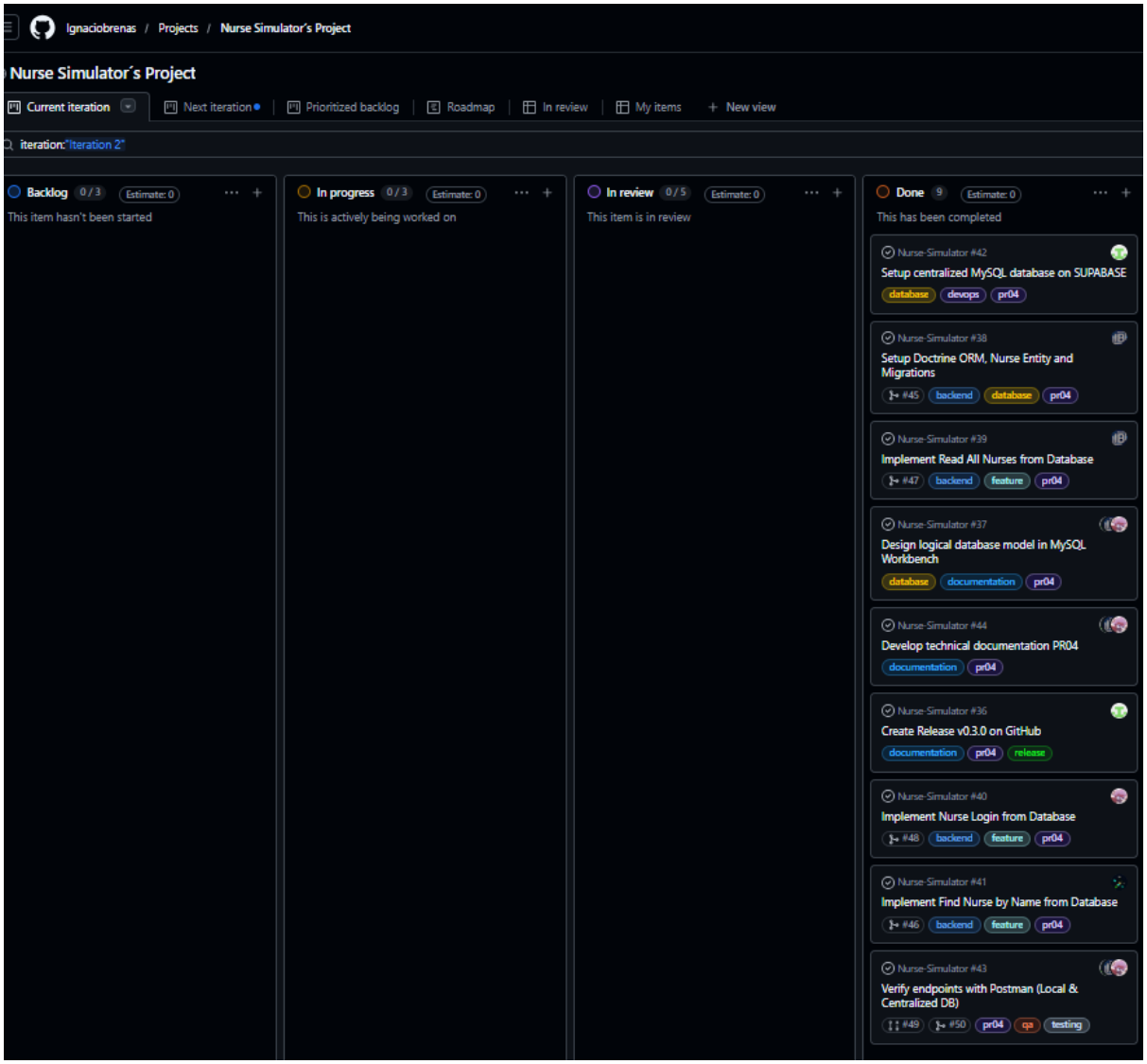


Figura 1. Tablero de tareas de la pr04.

2.2.2. Issues y ramas

Listado de issues de la pr04 y su rama asociada.

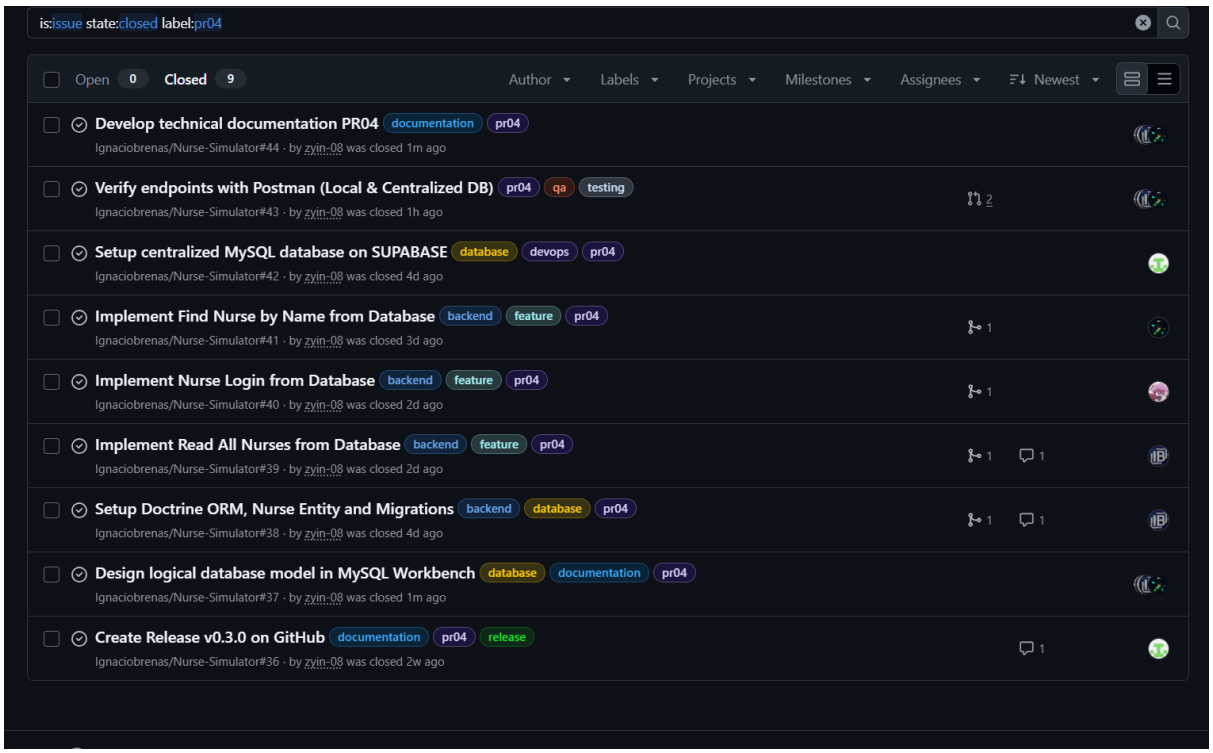
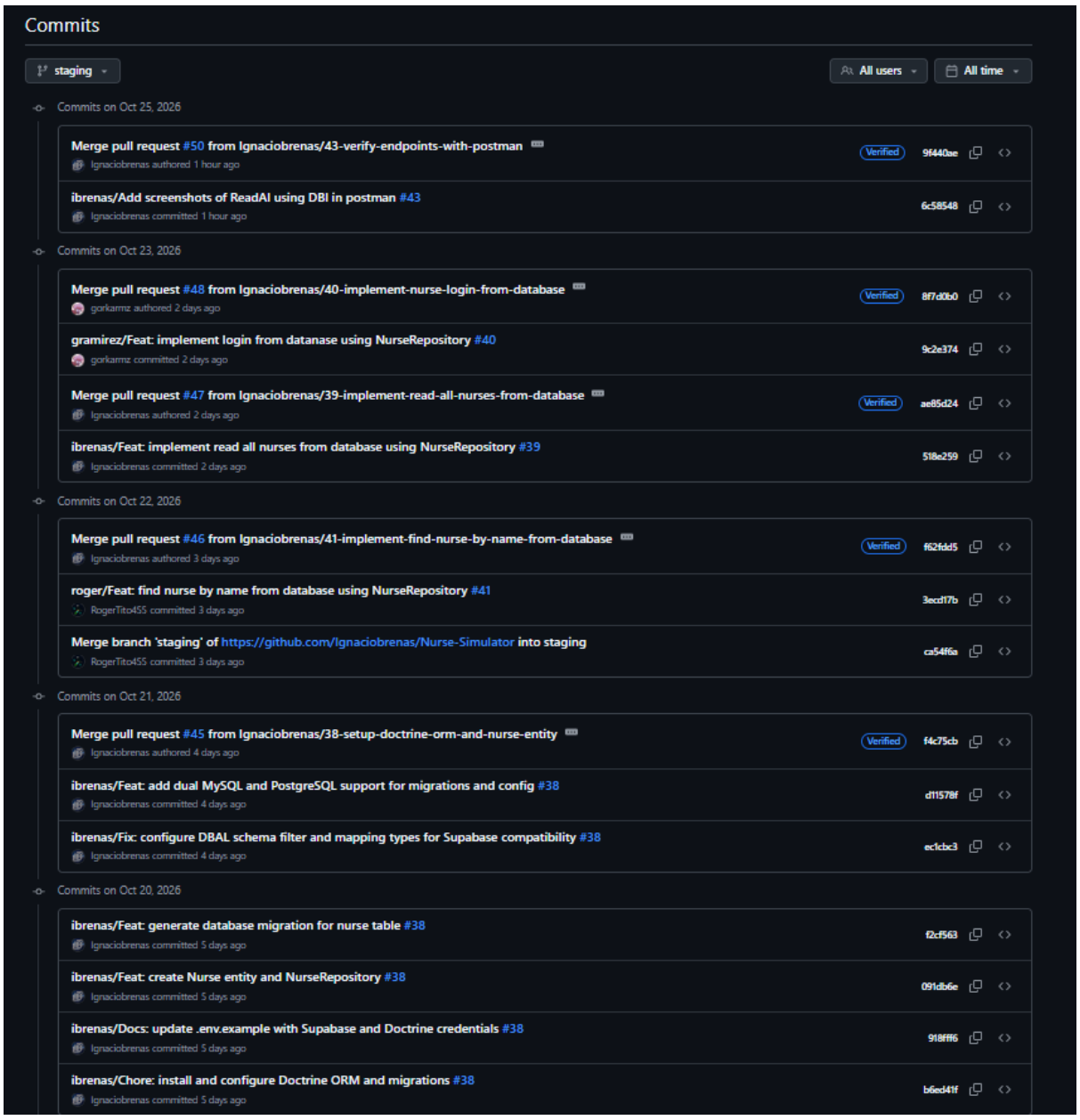




Figura 2. Issues y ramas de la pr04.

2.2.3. Pull requests y commits

Pull requests revisadas y fusionadas, con el historial de commits.



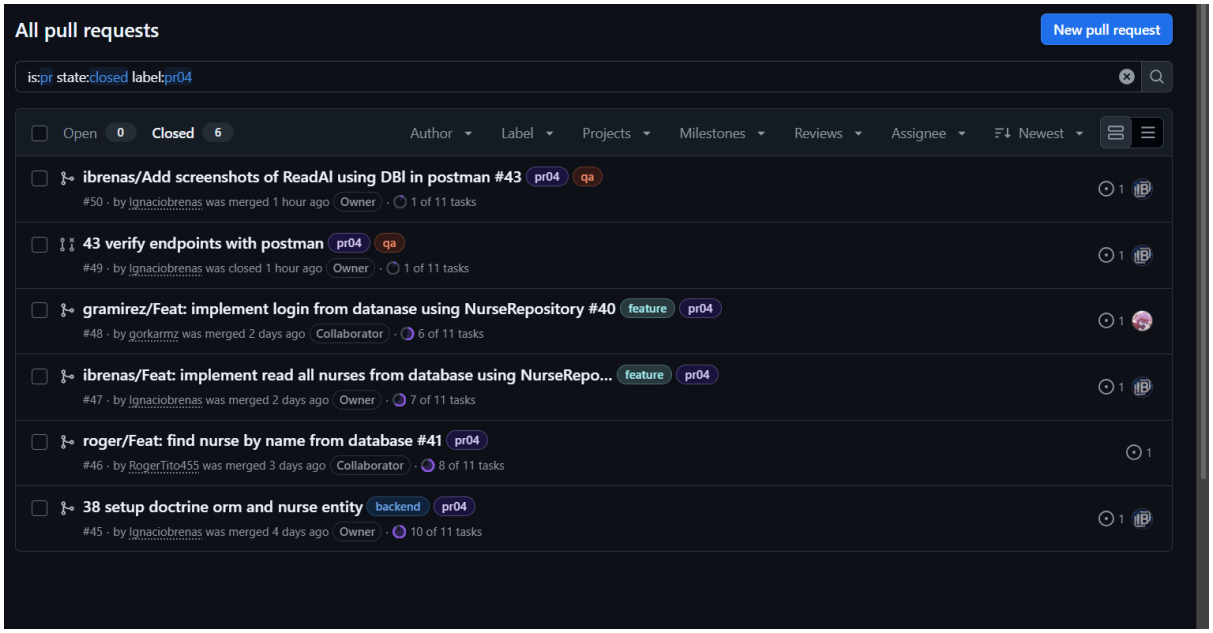


Figura 3. Pull requests y commits de la pr04.

3. Parte individual

Cada miembro del equipo realizó por separado, en su propio repositorio, la parte individual de la pr04: configurar Doctrine, crear la entidad `Nurse` y crear su repositorio `NurseRepository`.

3.1. Configuración y parametrización de Doctrine

3.1.1. Instalación

Se instaló Doctrine ORM en el proyecto Symfony de cada repositorio individual, de forma que el framework pueda mapear clases PHP a tablas de la base de datos.

3.1.2. Parametrización de la conexión

Se parametrizó la conexión de Doctrine con la base de datos PostgreSQL. Las credenciales se definen en el entorno local de cada miembro y no se suben al repositorio; en `.env.example` queda la plantilla que ya usábamos desde la pr03.

3.2. Entidad Nurse

Se creó la entidad `Nurse`, que representa a un enfermero y sustituye a los registros que antes vivían en `nurses.json`.

Doctrine la utiliza para generar y gestionar la tabla correspondiente en la base de datos.

3.3. Repositorio NurseRepository

Se creó `NurseRepository`, la clase que centraliza las consultas sobre la entidad `Nurse`. Es la que utilizan después los controladores para obtener los datos en lugar de leer el fichero JSON.

3.4. Enlaces a los repositorios individuales

En cada repositorio se puede comprobar con claridad la configuración y parametrización de Doctrine, la entidad `Nurse` y `NurseRepository`.

Integrante	Usuario de GitHub	Repositorio individual
Ignacio Breñas Muñoz	Ignaciobrenas	https://github.com/Ignaciobrenas/Ignacio-Nurse-Simulator
Christopher Roger Tito Farez	RogerTito455	https://github.com/RogerTito455/Nurse-Simulator-Individual
Zehao Yin	zyin-08	https://github.com/zyin-08/Nurse_MySQL_Database
Gorka Ramirez	gorkarmz	https://github.com/gorkarmz/Gorka-Nurse-Simulator

4. Parte grupal

4.1. Configuración de Doctrine en el repositorio grupal

Tras la parte individual, se configuró lo mismo en el repositorio grupal: Doctrine instalado y parametrizado, la entidad `Nurse` y `NurseRepository`. A partir de ahí, todos los integrantes trabajan sobre esta misma base.

https://github.com/Ignaciobrenas/Nurse-Simulator

4.2. Base de datos en Supabase (PostgreSQL)

4.2.1. Por qué Supabase

Para la base de datos centralizada hemos utilizado **Supabase** en lugar de un servicio de MySQL hosting como el que propone el enunciado. La razón es sencilla: ya hemos trabajado antes con esta plataforma y nos resulta cómoda, lo que nos permite centrarnos en la integración con Symfony y no en aprender una herramienta nueva.

4.2.2. PostgreSQL también en local

Supabase está basado en **PostgreSQL**. Para evitar conflictos entre entornos, cada integrante ha configurado también su base de datos local con PostgreSQL. Así, el motor, los tipos de datos y el comportamiento son los mismos en local y en el servidor, y el código funciona igual en ambos sin cambios.

-Aunque hemos hecho polimórfica la primera migración para que pueda usarse en **MySQL** también.

4.3. Adaptación de los endpoints

Los tres endpoints de la pr03 se han adaptado para que, en lugar de leer de `nurses.json`, obtengan la información de la base de datos configurada en Supabase. Cambia únicamente el origen de los datos.

4.3.1. Read all nurses: GET /nurse/index

Devuelve la lista completa de enfermeros registrados.

Ahora la obtiene de la base de datos a través de `NurseRepository`.

4.3.2. Nurse login: POST /nurse/login

Valida las credenciales de un enfermero.

La comprobación se hace ahora contra los enfermeros guardados en la base de datos.

4.3.3. Find nurse by name: GET /nurse/name/{name}

Busca a un enfermero por su nombre.

La búsqueda se realiza ahora sobre la base de datos mediante `NurseRepository`.

5. Diseño de la base de datos y pruebas

5.1. Diseño final de la base de datos

Diseño final de la base de datos que da soporte al proyecto.

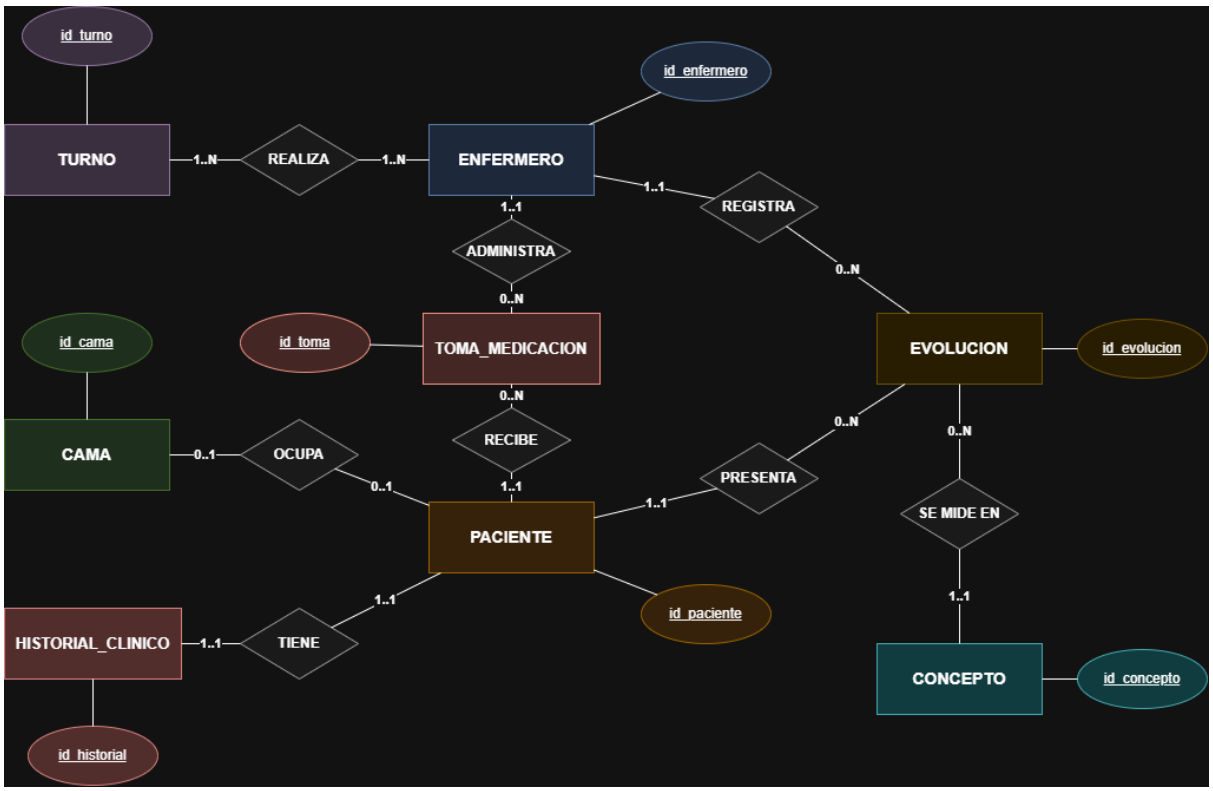


Figura 4. Diseño final de la base de datos.

ENFERMERO: Representa al personal sanitario del centro y gestiona sus credenciales de acceso al sistema (email, password_hash), datos personales y estado laboral (fecha_baja). Es el responsable de registrar evoluciones, administrar medicación y cubrir turnos.

PACIENTE: Almacena la información demográfica básica y de contacto de la persona ingresada o atendida en el hospital (nombre, fecha_nacimiento, sexo, telefono).

CAMA: Identifica el recurso físico de hospitalización dentro de una planta o área concreta (codigo, unidad) donde se ubica a un paciente durante su estancia.

TURNO: Define los bloques horarios de trabajo del personal (ej. mañana, tarde, noche) especificando su tramo horario (hora_inicio, hora_fin).

HISTORIAL_CLINICO: Contiene el expediente médico base y permanente del paciente (num_historia, fecha_apertura), recogiendo información crítica como sus alergias y el diagnostico principal.

CONCEPTO: Actúa como un catálogo o tabla maestra que define qué parámetros clínicos o constantes vitales se pueden medir (ej. temperatura, tensión arterial, glucemia) y en qué unidad_medida se expresan (°C, mmHg, mg/dL).

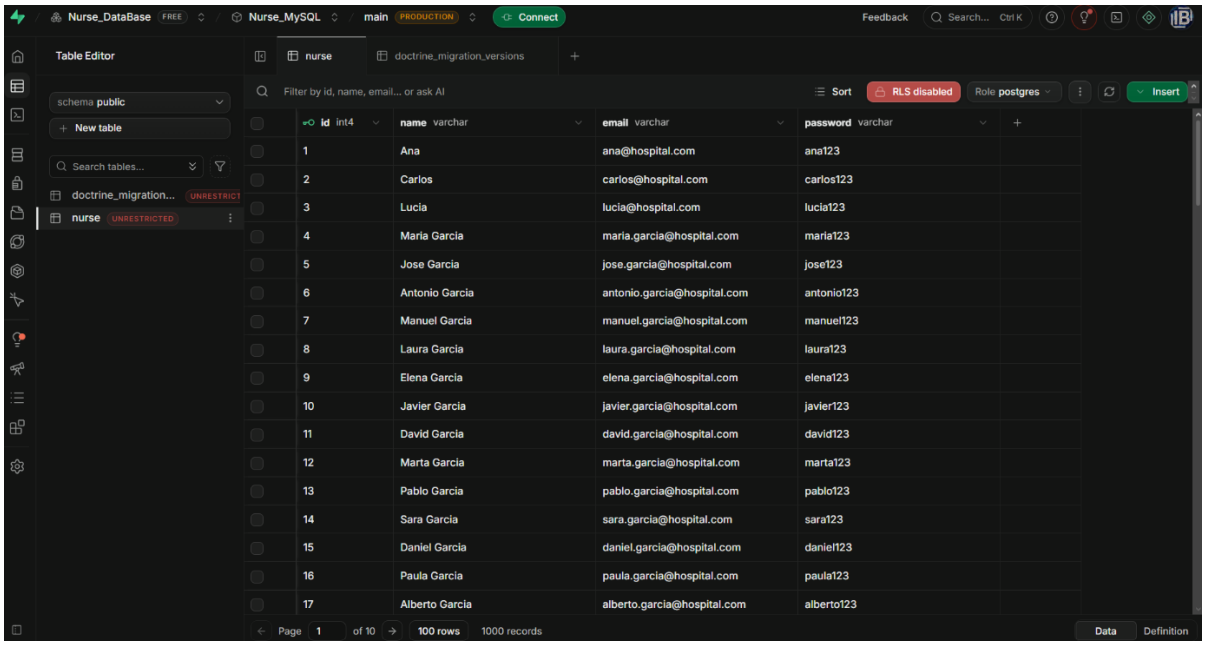
EVOLUCION: Es el registro clínico puntual realizado en una fecha_hora determinada donde el enfermero anota el valor medido de un concepto concreto (ej. "38.5" para el concepto "Temperatura") junto con sus observaciones sobre el estado del paciente.

T**OMA_MEDICACION**: Registra cada administración de un fármaco al paciente, detallando cuándo se dio (fecha_hora), qué cantidad (dosis), por qué via (oral, intravenosa, etc.) y su estado (administrada, pendiente, rechazada).

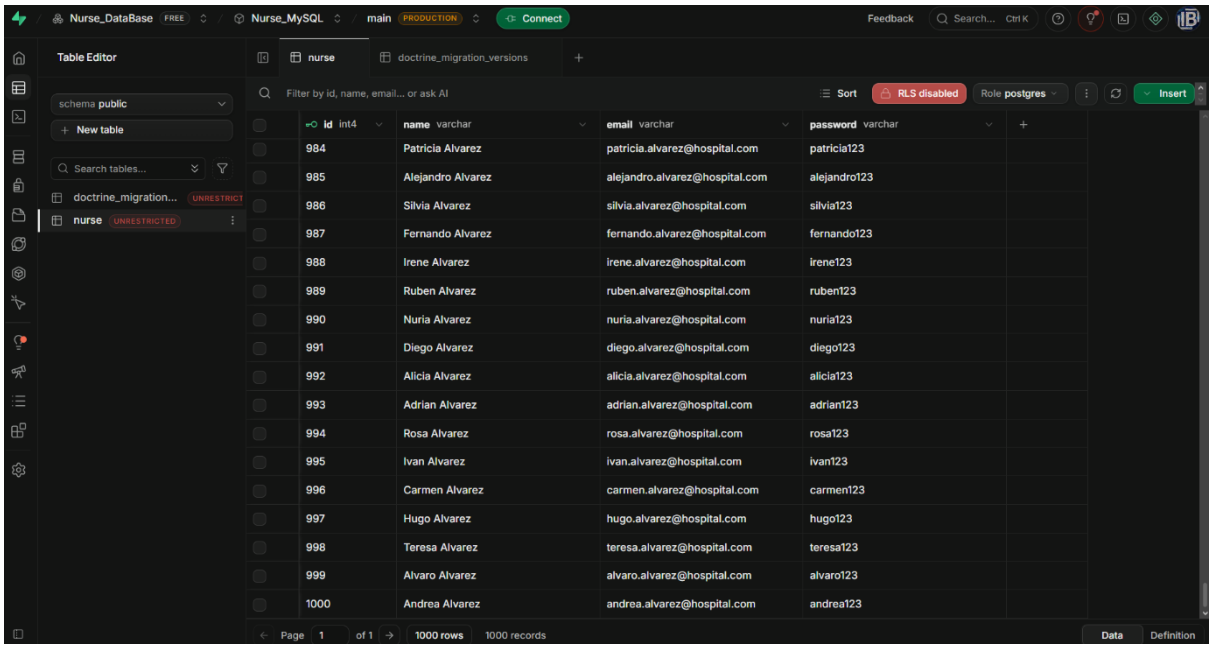
5.2. Comprobación con Postman

Comprobación del funcionamiento global de los endpoints con la base de datos de Supabase.

Esto es una captura desde **Supabase**. Rellenamos la base de datos remota con los mismos 1000 registros que en el **Nurses.json**

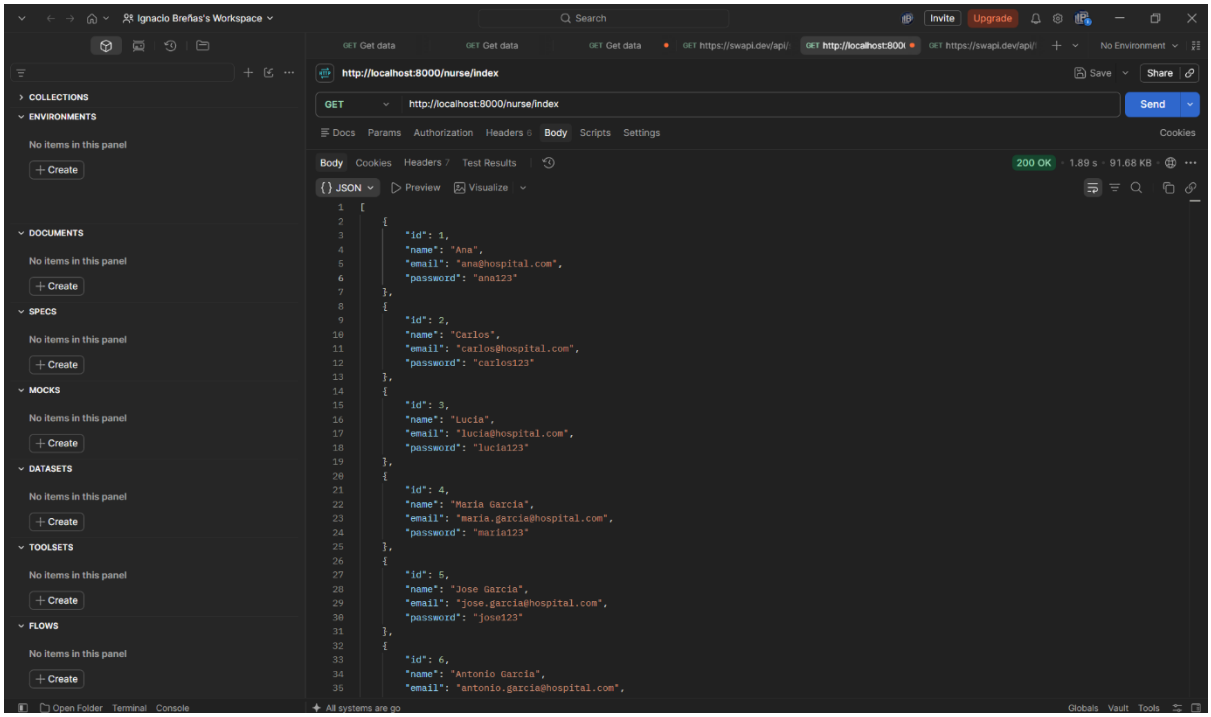


REGISTROS SUPABASE 1

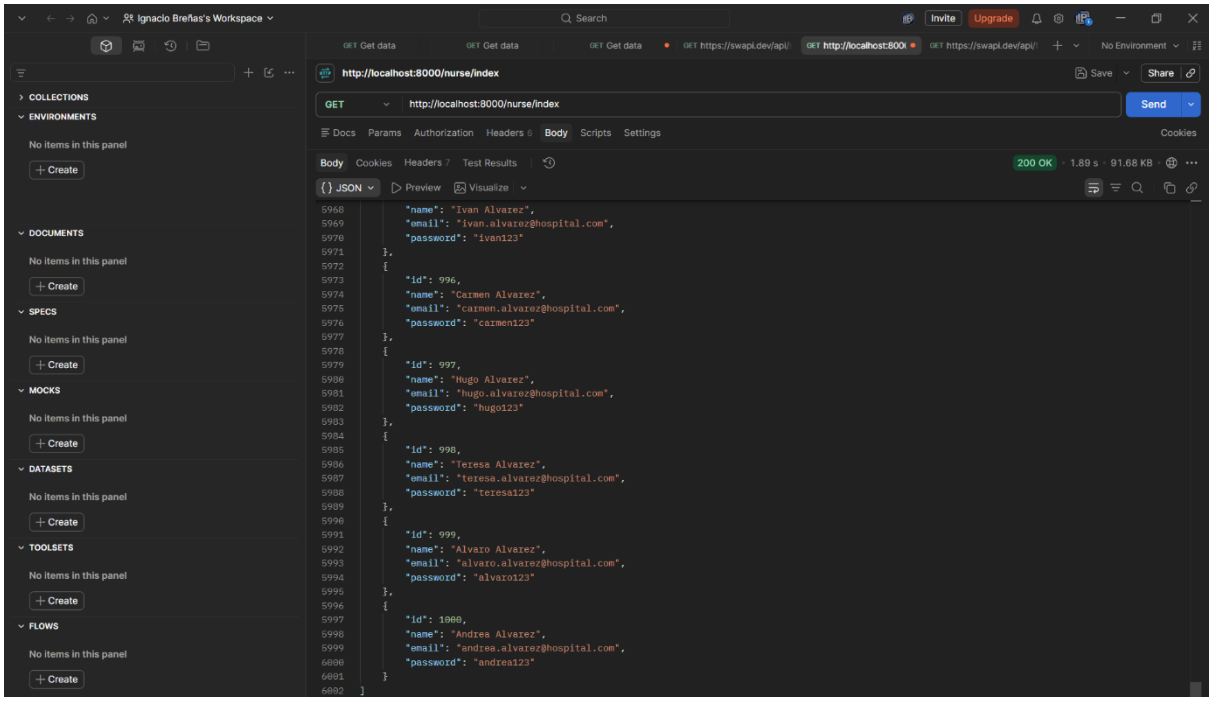


REGISTROS SUPABASE 2

5.2.1. GET /nurse/index



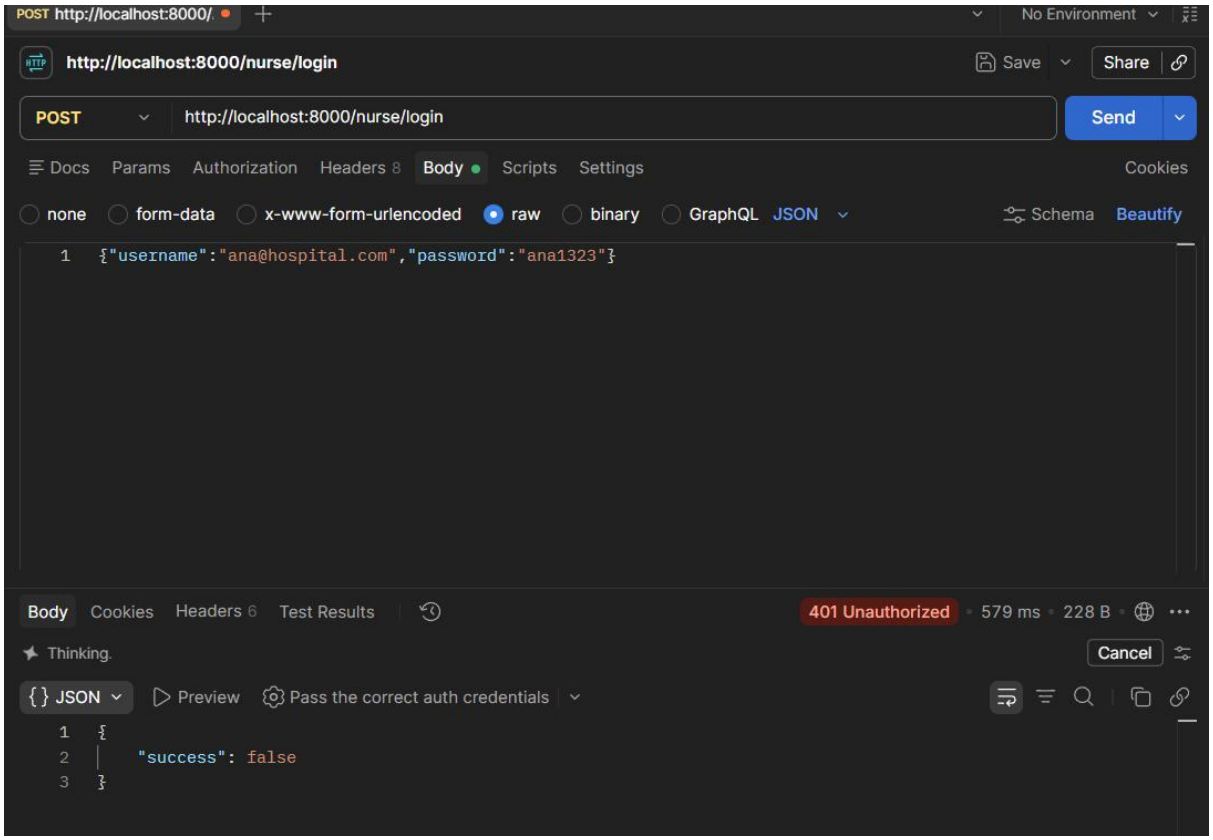
Postman-Read All 1



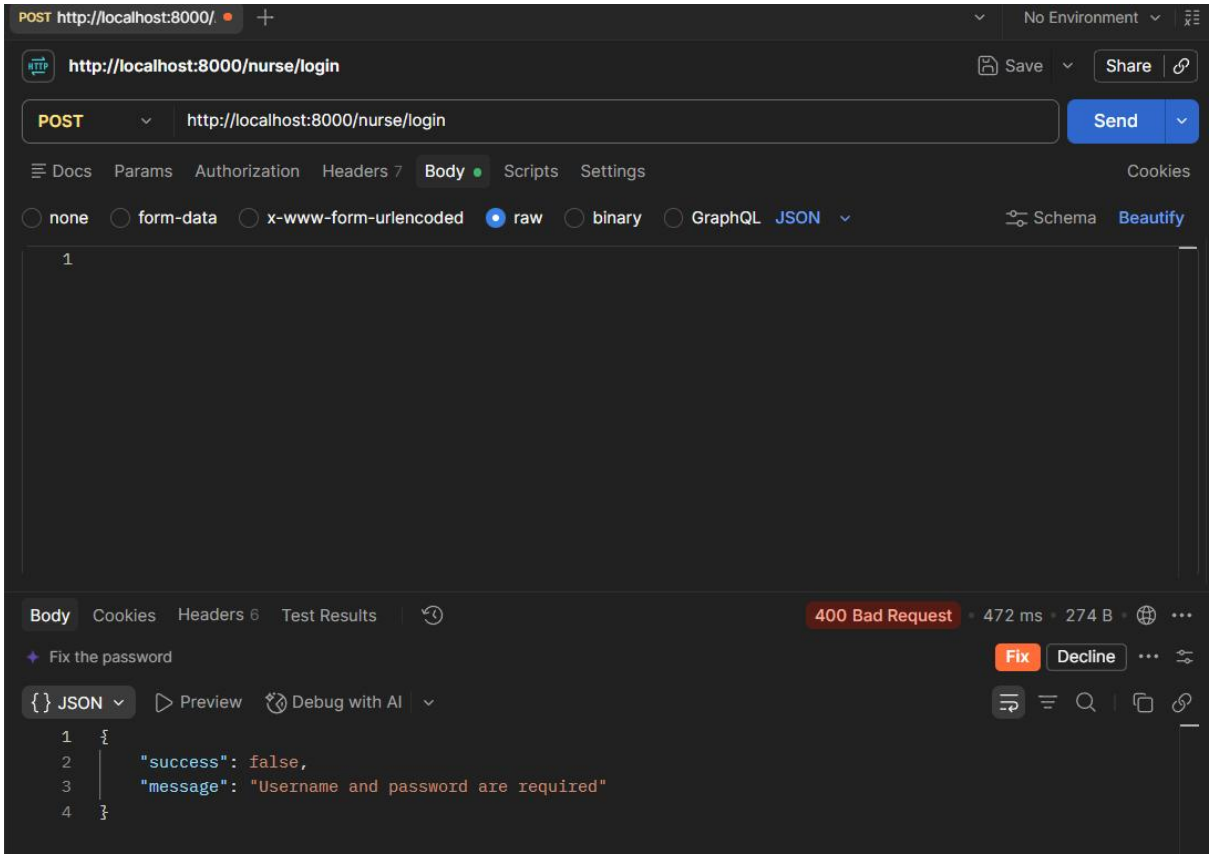
Postman-Read All 2

Listado de enfermeros desde la base de datos.

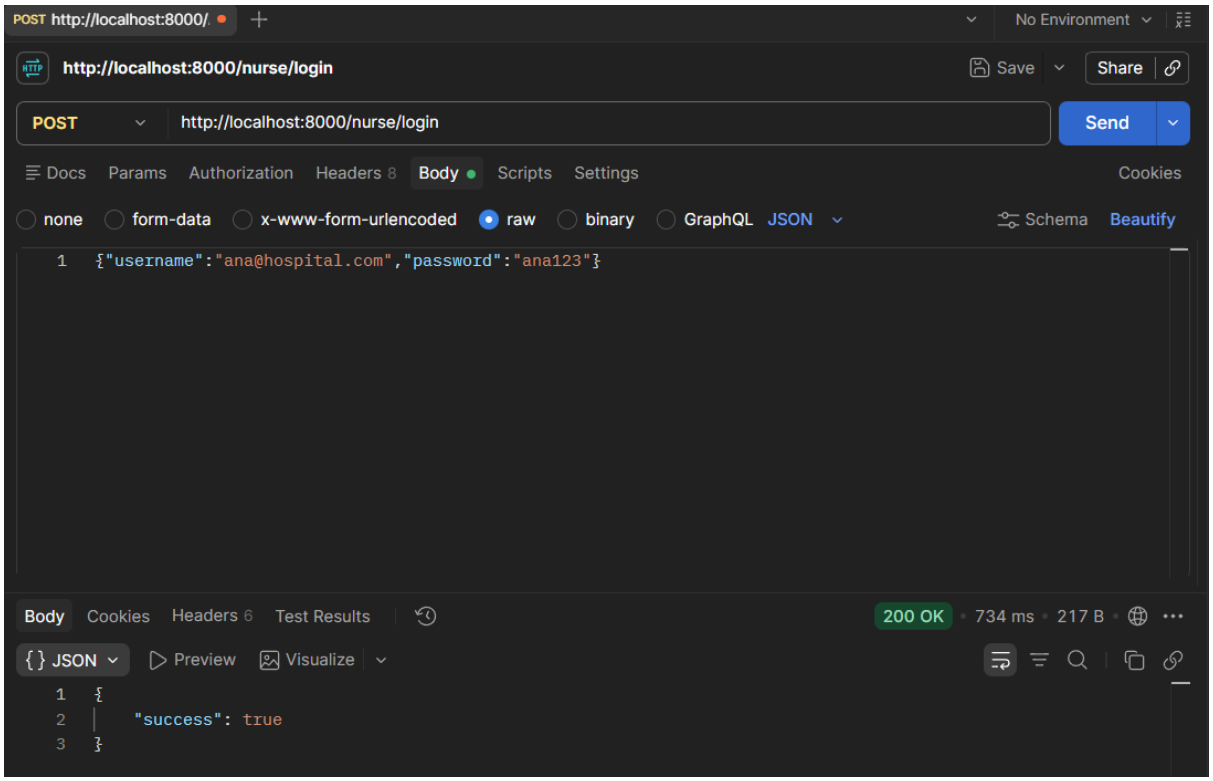
5.2.2. POST /nurse/login



Postman Login FALLIDO (bad credencials) 1

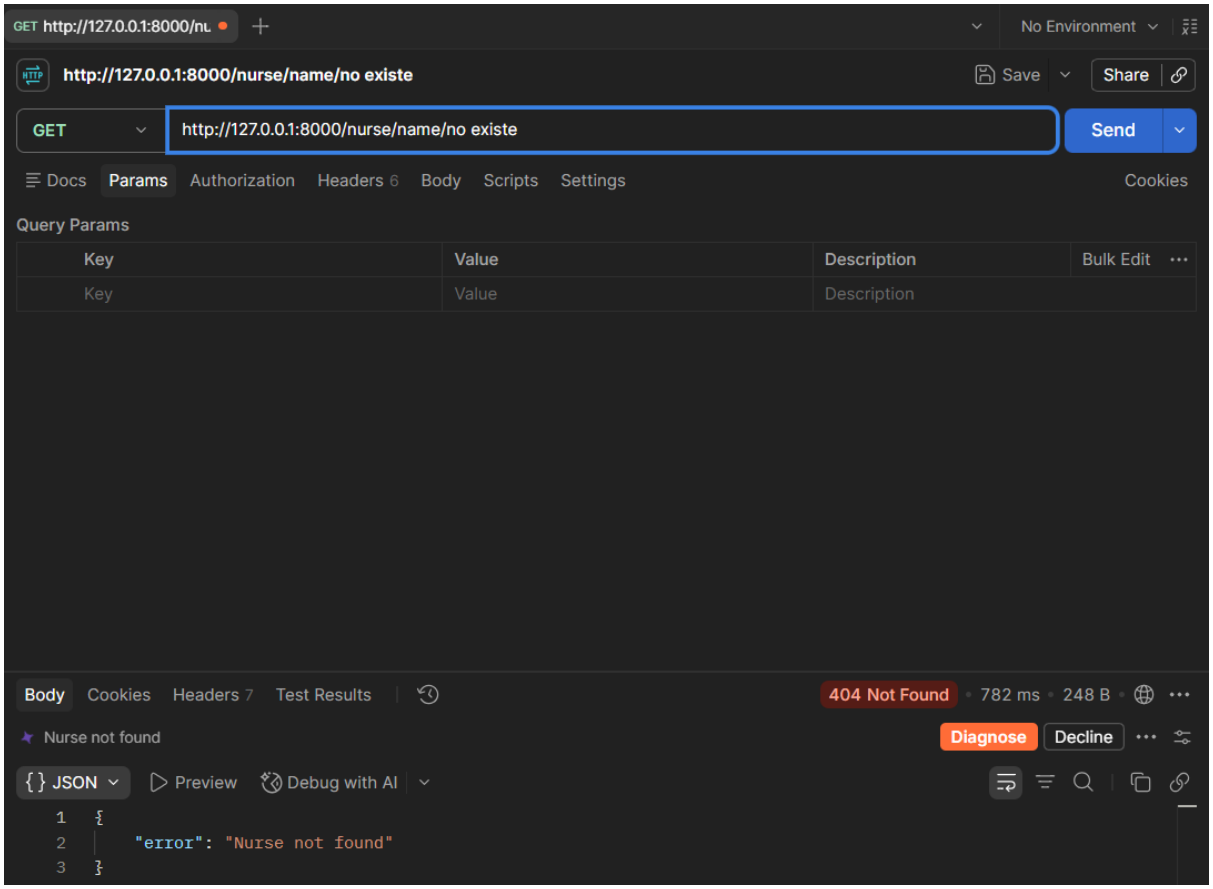


Postman Login (without credentials) 1

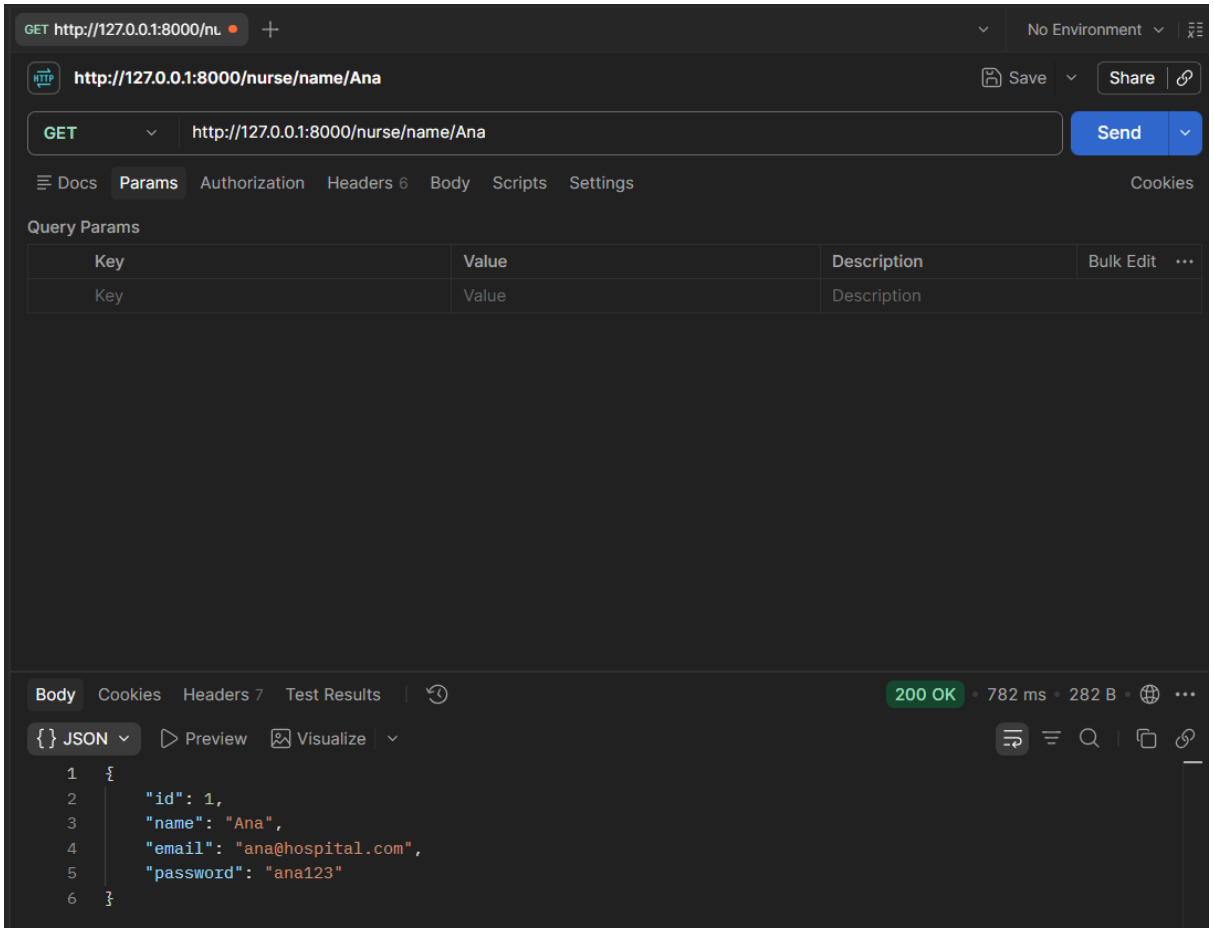


Postman Login (OK) 1

5.2.3. GET /nurse/name/{name}



Postman ReadByName (no existente) 1



Postman ReadByName (OK) 1

Búsqueda de un enfermero por nombre.